TASK 9: Spring MVC Application (Annotation-Based Configuration)

AIM

To develop a basic Spring MVC application using annotation-based configuration that handles user requests through a controller and displays employee details using the MVC architecture without XML configuration.

ALGORITHM

1. Start
2. Create Spring MVC project
3. Configure application using Java annotations (@Configuration, @ComponentScan)
4. Create **Employee Model class**
5. Create **Controller class** using @Controller
6. Map HTTP request using @RequestMapping
7. Pass employee data to view using Model
8. Create JSP/HTML view page
9. Run application on server (Tomcat)
10. End

Code :

```
<!DOCTYPE html>
<html>
<head>
   <title>Spring MVC Employee Dashboard</title>\
   <style>
      body {
         font-family: Arial;
    background: #f4f4f4;
```

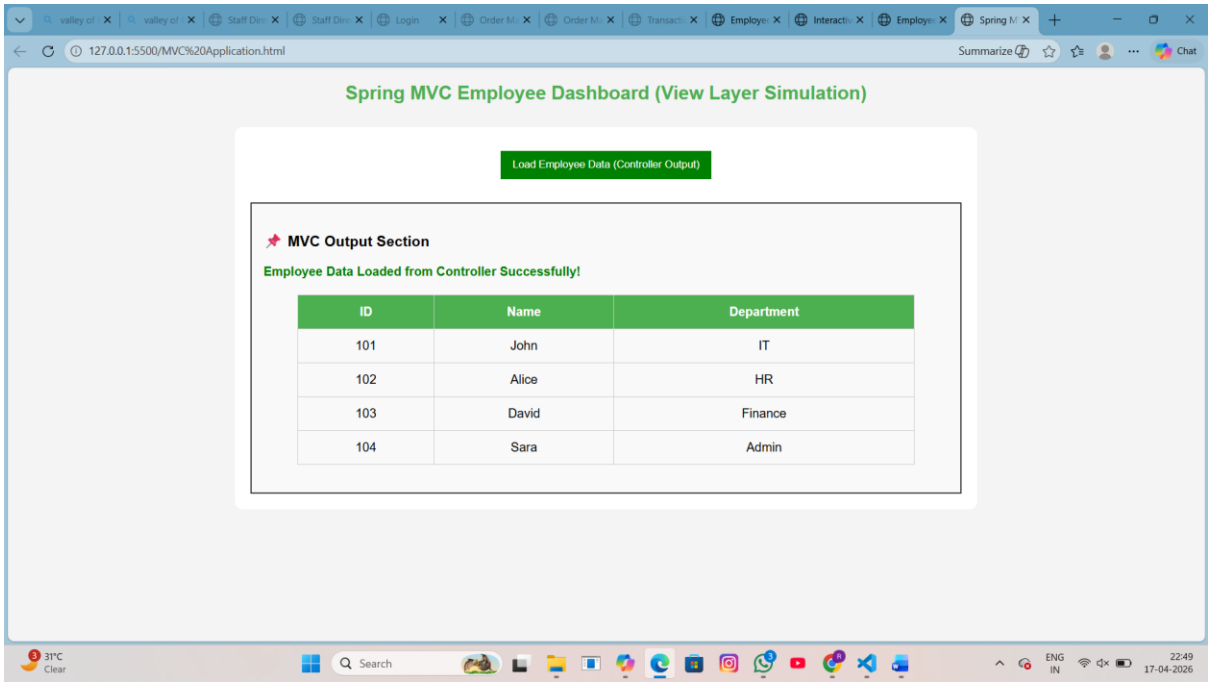
```
    text-align: center;
}
.container {
    width: 60%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 30px;
    border-radius: 10px;
}
h2 {
    color: #4CAF50;
}
table {
    width: 90%;
    margin: 20px auto;
    border-collapse: collapse;
}
th, td {
    border: 1px solid #ccc;
    padding: 12px;
    text-align: center;
}
th {
    background: #4CAF50;
    color: white;
}
button {
    padding: 10px 15px;
```

```
            margin: 10px;
            background: green;
            color: white;
            border: none;
            cursor: pointer;
        }
        #outputBox {
            margin-top: 20px;
            padding: 15px;
            border: 2px solid #333;
            background: #f9f9f9;
            text-align: left;
        }
        .success {
            color: green;
            font-weight: bold;
        }
    </style>
</head>
<body>
<h2>Spring MVC Employee Dashboard (View Layer Simulation)</h2>
<div class="container">
    <button onclick="loadEmployee()">Load Employee Data (Controller
Output)</button>
    <!-- OUTPUT PANEL -->
    <div id="outputBox">
       <h3>📌 MVC Output Section</h3>
        <p id="status">Click button to load employee data...</p>
        <table>
```

```
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Department</th>
                </tr>
            </thead>
            <tbody id="empTable"></tbody>
        </table>
    </div>
</div>
<script>
// ============================
// SIMULATED CONTROLLER DATA
// ============================
function loadEmployee() {
    // This simulates Controller in Spring MVC
    let employees = [
        { id: 101, name: "John", dept: "IT" },
        { id: 102, name: "Alice", dept: "HR" },
        { id: 103, name: "David", dept: "Finance" },
        { id: 104, name: "Sara", dept: "Admin" }
    ];
    let table = "";
    employees.forEach(emp => {
      table += `
            <tr>
                <td>${emp.id}</td>
                <td>${emp.name}</td>
```

```
            <td>${emp.dept}</td>
          </tr>
        `;
    });
    document.getElementById("empTable").innerHTML = table;
    document.getElementById("status").innerHTML =
        "<span class='success'>Employee Data Loaded from Controller Successfully!</span>";
}
</script>
</body>
</html>\
```

Output:



RESULT

A simple HTML view page was created to simulate Spring MVC output. It represents the View layer in MVC architecture, which displays employee details passed from the Controller. This demonstrates how data is shown in a structured format in real Spring MVC applications using JSP or HTML templates.